

COURSE 2A

C2-04 ADVANCED DAX & REPORTING

BEGINNER-FIRST TEACHING BOOK - RC4

Evaluation context -> DAX engineering -> report UX -> security -> lifecycle

THIS BOOK IS YOUR TEACHER

Power BI Desktop/Service behavior remains runtime evidence; internal teaching is self-sufficient.

This book is your teacher

Use one lesson at a time. New advanced concepts are still taught from first principles. Save a genuine attempt before opening protected review; do not skim ahead just because a topic name looks familiar.

Daily lesson loop

1) Why it matters -> 2) what you learn -> 3) prerequisite -> 4) picture it in your head -> 5) new words -> 6) watch / visual source when useful -> 7) follow with me once -> 8) expected result -> 9) why it worked -> 10) common beginner mistakes -> 11) recovery ladder -> 12) small fresh task -> 13) validate -> 14) teach it back -> 15) protected review + changed retry -> 16) evidence / mastery + exact next route.

Source rule

External videos are companions, not the course authority. If a useful source exists, open the exact link and return to the course task. If an external source would leak a solved capstone or add little value, the lesson explicitly uses an INTERNAL TEACHING ROUTE.

Attempt protection

Do the teacher example once. Then close it and complete the changed task from your own blank workbook/query/report/notebook. Protected review opens only after a saved attempt and is followed by a changed retry.

Mastery standard

You are finished only when you can reproduce the result, validate it, explain why it works, recover from a mistake and save the required evidence.

Contents / Index

Every lesson and actual page number in this **RC4** book

This book is your teacher	2
Contents / Index	3
Prerequisite and tool map	4
Practice, review and Gate route	5
DAX1	Evaluation context properly understood 6
DAX2	Row context vs filter context 10
DAX3	CALCULATE deeply understood 14
DAX4	Iterator functions 18
DAX5	Context transition 22
DAX6	Time intelligence 26
DAX7	Period comparison and rolling metrics 30
DAX8	Variables and readable DAX 34
DAX9	Advanced business measures 38
DAX10	Debugging incorrect measures 42
DAX11	Report storytelling and executive design 46
DAX12	Drillthrough, tooltips, bookmarks and interaction 50
DAX13	Accessibility and usability 54
DAX14	Row-level security 58
DAX15	Workspaces, sharing and lifecycle basics 62
Module mastery checklist + quick reference	66
Source / video registry	67

Prerequisite and tool map

What is assumed and where Desktop/Service execution is required

ASSUMED

Course 1 basic measures/visuals. C2-03 fact/dimension/star-schema, relationship/filter propagation and Date-table reasoning. New DAX concepts are still taught from first principles. You do not need to memorize function catalogs.

TOOLS

Power BI Desktop for core DAX/report/RLS practice. Provided model data/evidence workbooks will follow. Service access is optional until DAX15/lifecycle execution. No paid cloud feature will be a hidden requirement for internal lesson comprehension.

NEW DEPTH

Evaluation/row/filter context; CALCULATE; iterators; context transition; time/rolling measures; variables; business measure patterns; debugging; executive storytelling; interactions; accessibility; RLS; workspace/sharing/refresh lifecycle.

OPTIONAL-PATH BOUNDARY

C2-04 belongs to optional Senior/BI depth. It is useful for BI roles and PL-300 coverage, but it must not become a prerequisite for the direct Course 1 -> C2B -> C3 DA-to-DE path.

Practice, review and Gate route

Context first, evidence second, protected review third



DAX LESSON RULE

For DAX1-DAX10, test at detail, subtotal/grand-total and changed slicer/filter states. Save intermediate/base measures when useful. For report/security lessons, save interaction/accessibility/security/lifecycle test evidence.

MINI-LAB

After DAX1-DAX15, build an executive BI release: validated measure family, decision-first page, drillthrough/tooltip/bookmark, accessibility pass, RLS design/test and lifecycle checklist.

GATES

Fresh integrated Gate A follows Mini-Lab. B/C are protected changed full reassessments. One critical semantic/security/accessibility/reconciliation failure blocks pass despite other scores.

DAX1 - Evaluation context properly understood

1. Why it matters

The same measure returns different values in different visuals because DAX evaluates it inside the current context. If you treat a measure like an Excel cell formula, advanced DAX feels random.

2. What you are learning

Read a report cell as a filter-context question. Trace slicers, rows/columns, visual filters and relationship-propagated filters before changing the measure.

3. Prerequisite / what you already know

C2-03 star schema and relationship/filter-propagation basics; simple SUM measure.

4. Picture it in your head

Mental model

Question envelope: every visual cell carries slicers, row/column headers, page/report filters and relationship filters. The measure answers once inside that envelope; change the envelope and the same DAX can return a different correct scalar.

5. New words before we use them

Evaluation context: The conditions under which a DAX expression is evaluated. Filter context: The set of allowed values/rows created by report filters, slicers, visual fields, relationships and DAX filters. Scalar: A single value returned by a measure for the current evaluation.

6. Watch / visual source

Context diagram + tiny visual + three cell checks + 60-90 second explain-back. VIDEO SUPPORT: Chandoo DAX masterclass - 19:24-25:11

<https://www.youtube.com/watch?v=ID7TvkoQ6rY>

<https://learn.microsoft.com/en-us/dax/dax-overview>

After watching/opening: return to this lesson and reproduce the worked example from your own file/query/report. Save course-task evidence, not a screenshot of the source.

7. Follow with me once

Teacher demonstration - do this once with me, then close the example before your fresh task.

- 1 Define [Sales] = SUM(FactSales[NetSales]). By itself the measure has no fixed category.
- 2 Put DimProduct[Category] on matrix rows. Each row creates a different filter context such as Category = Hardware.
- 3 That dimension filter propagates through the active relationship to FactSales; the same [Sales] expression sees a different allowed fact-row set.
- 4 Add a Region slicer. Now each matrix cell combines Category + Region + any page/report filters.
- 5 Validate one cell by reproducing the same filters in a simple table or independent source control.

8. What you should see / be able to say

Expected result

You can predict why one measure returns different values across matrix rows without rewriting the measure.

9. Why it worked

Reasoning

The expression stayed the same; only the allowed data changed. Professional upgrade: Senior DAX debugging starts with context inspection. The first question is usually 'What filters reach this measure here?' rather than 'Which function should I add?'

10. Common beginner mistakes

1. Thinking the measure stores one permanent number. | 2. Blaming DAX before inspecting slicers/relationships. | 3. Calling every filter 'row context'. | 4. Debugging a complex dashboard instead of one tiny matrix cell.

11. If you get stuck - recovery ladder

Recovery

Create a table visual with only the dimension field and one measure. Remove extra slicers. Add filters back one at a time and state what rows should remain. Save the genuine attempt before review.

12. Now you do a small version alone

Fresh independent task

Fresh case: [Ticket Count] changes by Team and Priority. Draw the filter-context path for one matrix cell and prove it with a tiny table.

13. Validate your result

Validation

The predicted allowed dimension/fact rows match the observed measure for at least three cells.

14. Teach it back in your own words

Explain without reading

Explain evaluation context without using the phrase 'DAX is dynamic'.

15. Protected review + changed fresh retry

Attempt protection

Save context trace first. Review only after attempt; retry with Product + Month + Region.

16. Evidence / mastery + exact next route

Mastery evidence

Context diagram + tiny visual + three cell checks + 60-90 second explain-back. VIDEO SUPPORT: Chandoo DAX masterclass - 19:24-25:11 <https://learn.microsoft.com/en-us/dax/dax-overview> After saving evidence, continue to DAX2.

DAX2 - Row context vs filter context

1. Why it matters

Row context and filter context sound similar but behave differently. Confusing them causes calculated columns, iterators and measures to return results that look inconsistent.

2. What you are learning

Recognize row context in calculated columns/iterators; recognize filter context in measures/report cells; know that row context does not automatically filter a separate aggregation expression.

3. Prerequisite / what you already know

DAX1 evaluation/filter context and basic calculated-column awareness.

4. Picture it in your head

Mental model

Flashlight vs clipboard: row context points at the current row while an iterator/calculated column is working; filter context is the allowed-value set that measures see. They can interact, but they are not interchangeable.

5. New words before we use them

Row context: The current row being evaluated, common in calculated columns and iterators. Calculated column: A model column computed row by row and stored in the model. Measure: A calculation evaluated at query/report time under filter context.

6. Watch / visual source

Context classification table + slicer test + explain-back. VIDEO SUPPORT: Chandoo DAX masterclass - 13:39-25:11

<https://www.youtube.com/watch?v=ID7TvkoQ6rY>

<https://learn.microsoft.com/en-us/dax/dax-overview>

After watching/opening: return to this lesson and reproduce the worked example from your own file/query/report. Save course-task evidence, not a screenshot of the source.

7. Follow with me once

Teacher demonstration - do this once with me, then close the example before your fresh task.

- 1 In FactSales, calculated column `LineAmount = FactSales[Quantity] * FactSales[UnitPrice]` uses the current FactSales row.
- 2 Measure `[Sales] = SUM(FactSales[LineAmount])` does not use one current row; it aggregates rows allowed by filter context.
- 3 If you write a calculated column that references a naked aggregate expression, do not assume the current row becomes a filter automatically.
- 4 Compare a matrix measure to a calculated column value and state when each is evaluated.
- 5 Use measures for interactive aggregations and calculated columns only when a stored row-level attribute is justified.

8. What you should see / be able to say

Expected result

You can label whether an expression is using row context, filter context or both and predict whether user interactions change it.

9. Why it worked

Reasoning

The two contexts are separated by purpose: current-row evaluation versus allowed-set evaluation. Professional upgrade: Model design and DAX design work together. Prefer measures for reusable interactive business metrics; justify stored calculated columns by modeling need.

10. Common beginner mistakes

1. Saying measures have a 'current row' by default. | 2. Creating calculated columns for every KPI. | 3. Assuming row context automatically filters SUM of the same table. | 4. Treating stored columns as interactive calculations.

11. If you get stuck - recovery ladder

Recovery

Ask two questions: Am I evaluating one row? What filters define the allowed set? Write answers before editing DAX. Save the genuine attempt before review.

12. Now you do a small version alone

Fresh independent task

Fresh case: classify five expressions as measure, calculated column or iterator use and explain the context each sees.

13. Validate your result

Validation

Change a slicer and prove which calculation changes and which stored value does not.

14. Teach it back in your own words

Explain without reading

Explain row context as 'the row I am standing on' and filter context as 'the rows I am allowed to see'.

15. Protected review + changed fresh retry

Attempt protection

Attempt first. Changed retry uses customer-level calculated classification plus report measure.

16. Evidence / mastery + exact next route

Mastery evidence

Context classification table + slicer test + explain-back. VIDEO SUPPORT: Chandoo DAX masterclass - 13:39-25:11
<https://learn.microsoft.com/en-us/dax/dax-overview> After saving evidence, continue to DAX3.

DAX3 - CALCULATE deeply understood

1. Why it matters

CALCULATE is central to DAX because it changes the filter context in which an expression is evaluated. Memorizing formulas without this mental model creates fragile measures.

2. What you are learning

Read CALCULATE as 'evaluate this expression under modified filters'; distinguish adding/overwriting filters, KEEPFILTERS behavior conceptually, and relationship/filter modifiers.

3. Prerequisite / what you already know

DAX1 filter context and DAX2 context types.

4. Picture it in your head

Mental model

Filter rewrite desk: CALCULATE starts with the current filter context, then adds, replaces or removes specified filters before re-evaluating the expression. Narrate exactly which filter changed before trusting the result.

5. New words before we use them

CALCULATE: Evaluates an expression in a modified filter context. Filter modifier: A DAX function/argument used to remove, keep or alter filter/relationship behavior. KEEPFILTERS: Changes filter application so new filters intersect with existing filters instead of replacing the same-column filter.

6. Watch / visual source

Base/derived measures + filter trace + two validation cells + explain-back. VIDEO SUPPORT: Chandoo DAX masterclass - 34:18-46:34

<https://www.youtube.com/watch?v=ID7TvkoQ6rY>

<https://learn.microsoft.com/en-us/dax/calculate-function-dax>

After watching/opening: return to this lesson and reproduce the worked example from your own file/query/report. Save course-task evidence, not a screenshot of the source.

7. Follow with me once

Teacher demonstration - do this once with me, then close the example before your fresh task.

- 1 Start [Sales] = SUM(FactSales[NetSales]).
- 2 Create [Blue Sales] = CALCULATE([Sales], DimProduct[Color] = "Blue").
- 3 In a visual already filtered to Red, the CALCULATE filter on the same Color column normally replaces that Color filter unless KEEPFILTERS is used.
- 4 Put Category and Color in a tiny matrix. Predict [Sales] and [Blue Sales] before running.
- 5 Validate [Blue Sales] with an independent filtered table; then test CALCULATE with a different Region filter.

8. What you should see / be able to say

Expected result

You can describe exactly which filter is added/replaced and why the base measure is reused.

9. Why it worked

Reasoning

CALCULATE changes the environment, not the definition of [Sales]. Professional upgrade: CALCULATE makes measure branching powerful: define trusted base measures once, then create business variants by changing context deliberately.

10. Common beginner mistakes

1. Reading CALCULATE as 'do an advanced calculation'. | 2. Copying filters into every base measure instead of reusing measures. | 3. Ignoring same-column overwrite behavior. | 4. Using FILTER(table,...) for simple Boolean filters without reason.

11. If you get stuck - recovery ladder

Recovery

Say aloud: 'Evaluate [expression] after changing these filters...' List current filter and new filter on each affected column. Save the genuine attempt before review.

12. Now you do a small version alone

Fresh independent task

Fresh case: create [Premium Segment Sales] and [Premium East Sales]; predict behavior when the visual already filters Segment.

13. Validate your result

Validation

Tiny matrix/source checks match predictions in filtered and unfiltered cases.

14. Teach it back in your own words

Explain without reading

Explain CALCULATE without mentioning syntax first.

15. Protected review + changed fresh retry

Attempt protection

Attempt first. Changed retry applies a Status filter and a Region filter to a different base measure.

16. Evidence / mastery + exact next route

Mastery evidence

Base/derived measures + filter trace + two validation cells + explain-back. VIDEO SUPPORT: Chandoo DAX masterclass - 34:18-46:34 <https://learn.microsoft.com/en-us/dax/calculate-function-dax> After saving evidence, continue to DAX4.

DAX4 - Iterator functions

1. Why it matters

SUM cannot evaluate an expression such as Quantity * UnitPrice row by row unless that value already exists as a column. Iterators solve this by visiting rows of a table expression.

2. What you are learning

Use SUMX/AVERAGEX conceptually; state iterator table/grain; distinguish iterator row context from final aggregation; avoid iterating unnecessarily large/wrong-grain tables.

3. Prerequisite / what you already know

DAX2 row context and simple measures.

4. Picture it in your head

Mental model

Assembly line: an iterator takes a table, visits one row at a time, evaluates an expression at that row, then aggregates the produced values. If the input row grain is wrong, SUMX faithfully totals the wrong thing.

5. New words before we use them

Iterator: A function ending commonly in X that evaluates an expression for rows of a table, then aggregates results. Table expression: An expression returning a table of rows for the iterator to visit. SUMX: Iterator that evaluates an expression per row and sums the resulting values.

6. Watch / visual source

Iterator measure + row-grain statement + manual check + explain-back. VIDEO: intentionally book-led for this lesson

<https://learn.microsoft.com/en-us/dax/sumx-function-dax>

After watching/opening: return to this lesson and reproduce the worked example from your own file/query/report. Save course-task evidence, not a screenshot of the source.

7. Follow with me once

Teacher demonstration - do this once with me, then close the example before your fresh task.

- 1 Use [Gross Sales] = SUMX(FactSales, FactSales[Quantity] * FactSales[UnitPrice]).
- 2 The iterator table is FactSales; current row is one sales line.
- 3 For each allowed fact row, calculate Quantity * UnitPrice; SUMX then adds the row results.
- 4 Filter context from Category/Region still determines which FactSales rows enter the iterator.
- 5 Validate one small filtered selection by manually summing 3-5 visible rows.

8. What you should see / be able to say

Expected result

You can state both the iterator row grain and the outer filter context.

9. Why it worked

Reasoning

SUMX creates row context for the expression while respecting the filtered table rows supplied to it. Professional upgrade: Iterators are powerful but should be used at a deliberate table grain. Measure and model design often determine whether a simple aggregate or iterator is clearer/faster.

10. Common beginner mistakes

1. Iterating a dimension when the expression belongs at fact grain. | 2. Using SUMX when SUM of an existing additive column is enough. | 3. Forgetting the table expression may already be filtered. | 4. Assuming X functions automatically fix wrong grain.

11. If you get stuck - recovery ladder

Recovery

Write: 'SUMX visits one row of ____ where one row means ____.' Then state which report filters restrict that table. Save the genuine attempt before review.

12. Now you do a small version alone

Fresh independent task

Fresh case: [Line Cost] = Quantity * UnitCost plus per-line HandlingFee; build [Total Cost] without creating an unnecessary stored column.

13. Validate your result

Validation

Manual filtered row calculation matches measure for a tiny selection.

14. Teach it back in your own words

Explain without reading

Explain SUM versus SUMX using 'sum a column' versus 'calculate each row, then sum'.

15. Protected review + changed fresh retry

Attempt protection

Attempt first. Changed retry uses AVERAGEX over customer-level values rather than transaction rows.

16. Evidence / mastery + exact next route

Mastery evidence

Iterator measure + row-grain statement + manual check + explain-back. VIDEO: intentionally book-led for this lesson
<https://learn.microsoft.com/en-us/dax/sumx-function-dax> After saving evidence, continue to DAX5.

DAX5 - Context transition

1. Why it matters

Context transition explains several DAX results that otherwise seem magical. It occurs when CALCULATE converts row context into filter context; model measures invoked in row context receive this behavior automatically.

2. What you are learning

Recognize where row context exists, when context transition happens, why CALCULATE without explicit filters can matter, and why careless calculated-column use can be expensive/confusing.

3. Prerequisite / what you already know

DAX2 row context, DAX3 CALCULATE and DAX4 iterators.

4. Picture it in your head

Mental model

Badge scanner: inside row context, CALCULATE or a measure reference can turn current-row values into filters before aggregation. This context transition is invisible unless you deliberately trace which row values became filters.

5. New words before we use them

Context transition: Conversion of existing row context into equivalent filter context for expression evaluation. Measure reference: Calling an existing model measure by name inside another expression. Implicit transition: Automatic context transition when a model measure is evaluated in row context.

6. Watch / visual source

Transition diagram + iterator/measure example + validation + explain-back. VIDEO SUPPORT: Chandoo DAX masterclass - 34:18-46:34 for CALCULATE context; book-led transition

<https://www.youtube.com/watch?v=ID7TvkoQ6rY>

<https://learn.microsoft.com/en-us/training/modules/dax-power-bi-modify-filter/>

After watching/opening: return to this lesson and reproduce the worked example from your own file/query/report. Save course-task evidence, not a screenshot of the source.

7. Follow with me once

Teacher demonstration - do this once with me, then close the example before your fresh task.

- 1 Suppose an iterator visits DimCustomer one customer row at a time.
- 2 When the iterator expression references model measure [Sales], the current customer row can become an equivalent filter context for that measure through context transition.
- 3 Contrast this with a raw aggregate expression placed in row context without CALCULATE; do not assume it receives the same filtering.
- 4 Build a tiny customer table with [Sales] and compare against source/grouped totals.
- 5 Use context transition because the business grain demands it, not as a ritual around every measure.

8. What you should see / be able to say

Expected result

You can identify the row context being transitioned and which columns become filters.

9. Why it worked

Reasoning

The current row becomes filter context, allowing the aggregate measure to evaluate for that row's entity. Professional upgrade: Understanding transition is more important than memorizing edge cases. Use small test tables and DAX Query View/visuals to inspect behavior.

10. Common beginner mistakes

1. Calling context transition every time filters change. | 2. Wrapping every measure in CALCULATE unnecessarily. | 3. Forgetting relationship/model design affects resulting filters. | 4. Using calculated columns for dynamic report metrics.

11. If you get stuck - recovery ladder

Recovery

Locate the row context first. If none exists, there is nothing to transition. Then identify the CALCULATE or model-measure evaluation. Save the genuine attempt before review.

12. Now you do a small version alone

Fresh independent task

Fresh case: iterate customers to calculate average customer revenue using a measure; explain where transition happens.

13. Validate your result

Validation

Compare selected customer values to independently filtered base measure results.

14. Teach it back in your own words

Explain without reading

Explain context transition as turning 'the row I am on' into 'filter the model to this row's values'.

15. Protected review + changed fresh retry

Attempt protection

Attempt first. Changed retry uses Product rows and a margin measure.

16. Evidence / mastery + exact next route

Mastery evidence

Transition diagram + iterator/measure example + validation + explain-back. VIDEO SUPPORT: Chandoo DAX masterclass - 34:18-46:34 for CALCULATE context; book-led transition <https://learn.microsoft.com/en-us/training/modules/dax-power-bi-modify-filter/> After saving evidence, continue to DAX6.

DAX6 - Time intelligence

1. Why it matters

Time calculations fail when the model has incomplete/ambiguous dates or the measure shifts transaction rows instead of a governed calendar.

2. What you are learning

Use a dedicated Date table, define current period, shift/filter dates with current supported functions, and validate YTD/prior-period measures against explicit period boundaries.

3. Prerequisite / what you already know

C2-03 Date dimension and role relationships; DAX3 CALCULATE.

4. Picture it in your head

Mental model

Date conveyor belt: the current date set is shifted to a comparable date set and the same base measure runs again. The continuous Date table is the track; gaps, wrong relationships or incomplete periods derail the comparison.

5. New words before we use them

Time intelligence: Calculations that evaluate measures across related time periods. Date table: A governed calendar used as the time-filtering dimension. DATEADD: DAX function that shifts the current date/calendar context by a specified interval.

6. Watch / visual source

Date-table/role note + time measures + period controls + explain-back. VIDEO: intentionally book-led for this lesson

<https://learn.microsoft.com/en-us/power-bi/transform-model/desktop-time-intelligence>

After watching/opening: return to this lesson and reproduce the worked example from your own file/query/report. Save course-task evidence, not a screenshot of the source.

7. Follow with me once

Teacher demonstration - do this once with me, then close the example before your fresh task.

- 1 Start with a valid Date table related to FactSales OrderDate and base [Sales].
- 2 Prior Month pattern: `CALCULATE([Sales], DATEADD(DimDate[Date], -1, MONTH))` when using a date-column design that meets function requirements.
- 3 YTD can use supported time-intelligence patterns over the governed Date table; choose the function/pattern that matches calendar semantics.
- 4 Test January/first-period behavior and one month with no sales. Zero activity is not the same as missing calendar date.
- 5 Validate monthly values against a source pivot/SQL control and verify date-role choice (Order Date versus Ship Date).

8. What you should see / be able to say

Expected result

Time measure whose calendar, date role and boundary behavior are explicit.

9. Why it worked

Reasoning

The Date dimension defines the time set; `CALCULATE` evaluates the same trusted base measure under shifted/accumulated date context. Professional upgrade: Microsoft's current time-intelligence capabilities continue to evolve. Use current documentation, a governed calendar and explicit role assumptions rather than copying old formulas blindly.

10. Common beginner mistakes

1. Using text MonthName as the time engine. | 2. Relying on hidden auto-date in a governed model. | 3. Comparing ShipDate and OrderDate accidentally. | 4. Assuming all DATEADD selections are valid/contiguous without checking.

11. If you get stuck - recovery ladder

Recovery

12. Now you do a small version alone

Fresh independent task

Fresh case: create prior-week and YTD ticket measures using a governed Date table; document which fact date role is active.

13. Validate your result

Validation

At least three periods reconcile to an independent control including the first available period.

14. Teach it back in your own words

Explain without reading

Explain why time intelligence is a Date-table problem before it is a DAX-function problem.

15. Protected review + changed fresh retry

Attempt protection

Attempt first. Changed retry uses fiscal-period comparison rather than calendar month.

16. Evidence / mastery + exact next route

Mastery evidence

Date-table/role note + time measures + period controls + explain-back. VIDEO: intentionally book-led for this lesson
<https://learn.microsoft.com/en-us/power-bi/transform-model/desktop-time-intelligence> After saving evidence, continue to DAX7.

DAX7 - Period comparison and rolling metrics

1. Why it matters

YoY, MoM and rolling metrics can mislead when the comparison period is incomplete, denominator is blank/zero, or the rolling window includes the wrong dates.

2. What you are learning

Build comparison measures in layers: base -> prior period -> absolute difference -> percentage change; then build rolling-window measures with explicit window definition and completeness rules.

3. Prerequisite / what you already know

DAX6 time intelligence and DAX8 variables will later improve readability.

4. Picture it in your head

Mental model

Moving window frame: anchor on the current period, construct the exact prior/rolling date window, re-evaluate the base measure, then compare. State whether partial periods are allowed before the percent looks impressive.

5. New words before we use them

Period-over-period: Comparing a metric with another defined period such as prior month/year. Rolling window: A moving set of recent periods used to calculate a metric. Comparable period: A prior/current period valid for comparison under the business rule.

6. Watch / visual source

Layered measures + comparison/window contract + boundary checks + explain-back. VIDEO: intentionally book-led for this lesson

<https://learn.microsoft.com/en-us/dax/dateadd-function-dax>

After watching/opening: return to this lesson and reproduce the worked example from your own file/query/report. Save course-task evidence, not a screenshot of the source.

7. Follow with me once

Teacher demonstration - do this once with me, then close the example before your fresh task.

- 1 Define [Sales PY] from [Sales] using a current supported prior-year time pattern.
- 2 Define [Sales YoY] = [Sales] - [Sales PY].
- 3 Define [Sales YoY %] = DIVIDE([Sales YoY], [Sales PY]) so zero/blank denominator handling is explicit.
- 4 For rolling 3 months, define exactly which dates count and whether the current partial month is allowed. Use a date-window expression over DimDate.
- 5 Validate both current/prior totals and the exact first/last date in one rolling window.

8. What you should see / be able to say

Expected result

Layered comparison measures with visible prior value, delta, percentage and a documented rolling-window contract.

9. Why it worked

Reasoning

Each intermediate measure can be inspected independently, preventing one final percentage from hiding a bad denominator. Professional upgrade: Comparison measures should carry period-completeness and business-calendar rules in their metric contract, especially for executive reporting.

10. Common beginner mistakes

1. Calculating percentage before validating prior period. | 2. Comparing incomplete current month to full prior month. | 3. Calling last 90 days the same as rolling 3 calendar months. | 4. Returning 0 when comparison is undefined without agreement.

11. If you get stuck - recovery ladder

Recovery

Put Base, Prior, Delta and % side by side. Fix the first column where the values diverge from the expected period. Save the genuine attempt before review.

12. Now you do a small version alone

Fresh independent task

Fresh case: 4-week rolling resolved-ticket count plus week-over-week %; current week is incomplete.

13. Validate your result

Validation

Check exact window dates and independently recompute one comparison.

14. Teach it back in your own words

Explain without reading

Explain why rolling 3 months and last 90 days are not always identical.

15. Protected review + changed fresh retry

Attempt protection

Attempt first. Changed retry uses quarter-over-quarter margin rather than revenue.

16. Evidence / mastery + exact next route

Mastery evidence

Layered measures + comparison/window contract + boundary checks + explain-back. VIDEO: intentionally book-led for this lesson
<https://learn.microsoft.com/en-us/dax/dateadd-function-dax> After saving evidence, continue to DAX8.

DAX8 - Variables and readable DAX

1. Why it matters

Repeated expressions make measures harder to read and can cause repeated evaluation. Variables let you name important intermediate results and inspect logic more safely.

2. What you are learning

Use VAR/RETURN for meaningful intermediate values; separate base/prior/delta logic; debug by temporarily returning one variable; avoid variables that freeze a value earlier than intended context changes.

3. Prerequisite / what you already know

DAX3 CALCULATE and DAX7 layered measures.

4. Picture it in your head

Mental model

Labeled test points: VAR names meaningful intermediate results once, RETURN composes them, and debugging can temporarily return each variable. A variable keeps the value/context it had when defined - it is not a live spreadsheet cell.

5. New words before we use them

VAR: Defines a variable whose value is evaluated for use in the expression. RETURN: Specifies the expression returned after variable definitions. Measure branching: Building new measures by reusing trusted existing measures.

6. Watch / visual source

Before/after measure + intermediate checks + explain-back. VIDEO SUPPORT: Chandoo DAX masterclass - 25:12-34:17 measure reuse/DIVIDE context

<https://www.youtube.com/watch?v=ID7TvkoQ6rY>

<https://learn.microsoft.com/en-us/dax/best-practices/dax-variables>

After watching/opening: return to this lesson and reproduce the worked example from your own file/query/report. Save course-task evidence, not a screenshot of the source.

7. Follow with me once

Teacher demonstration - do this once with me, then close the example before your fresh task.

- 1 VAR SalesPriorYear = CALCULATE([Sales], DATEADD(DimDate[Date], -1, YEAR)).
- 2 VAR SalesChange = [Sales] - SalesPriorYear.
- 3 RETURN DIVIDE(SalesChange, SalesPriorYear).
- 4 During debugging, temporarily RETURN SalesPriorYear to inspect it; then restore the intended result.
- 5 Use names that express business meaning. Avoid v1, tmp2 unless the scope genuinely needs technical naming.

8. What you should see / be able to say

Expected result

A measure whose intermediate logic can be read top-to-bottom and independently inspected.

9. Why it worked

Reasoning

The prior-year result is calculated once, named and reused. Professional upgrade: Microsoft guidance notes variables can also reduce repeated evaluation and improve maintainability. Prefer clear measure branching plus variables for local intermediate results.

10. Common beginner mistakes

1. Creating a variable for every trivial literal. | 2. Using vague variable names. | 3. Defining a variable before a context-changing operation and expecting it to be re-evaluated later. | 4. Treating variables as global/shared across measures.

11. If you get stuck - recovery ladder

Recovery

Replace the final RETURN with one variable at a time. Compare its value in a tiny visual, then rebuild the final expression. Save the genuine attempt before review.

12. Now you do a small version alone

Fresh independent task

Fresh case: variance-to-target measure with Actual, Target, Variance and VariancePct variables.

13. Validate your result

Validation

Each intermediate variable matches an independently visible base measure.

14. Teach it back in your own words

Explain without reading

Explain why a variable improves both debugging and readability even when the final number is unchanged.

15. Protected review + changed fresh retry

Attempt protection

Attempt first. Changed retry refactors a rolling-period measure.

16. Evidence / mastery + exact next route

Mastery evidence

Before/after measure + intermediate checks + explain-back. VIDEO SUPPORT: Chandoo DAX masterclass - 25:12:34:17 measure reuse/DIVIDE context <https://learn.microsoft.com/en-us/dax/best-practices/dax-variables> After saving evidence, continue to DAX9.

DAX9 - Advanced business measures

1. Why it matters

A useful executive measure is not 'advanced' because it uses many functions. It is advanced when the business definition, denominator, context changes and exception behavior are explicit.

2. What you are learning

Build measure families such as contribution %, variance-to-target, weighted margin and semi-additive closing balance using measure branching and clear contracts.

3. Prerequisite / what you already know

DAX1-DAX8 and C2-03 model grain.

4. Picture it in your head

Mental model

Business-measure contract: write eligible population, numerator, denominator, exception rules, blank/zero policy and total behavior before DAX. The formula implements that contract; it does not invent the business definition.

5. New words before we use them

Contribution percentage: A part divided by an explicitly defined total/reference context. Semi-additive measure: A measure that can aggregate across some dimensions but not simply across time, such as end-of-period inventory balance. Reference context: The comparison filter context used for denominator/benchmark.

6. Watch / visual source

Measure family + contracts + subtotal/total tests + explain-back. VIDEO SUPPORT: Chandoo DAX masterclass - 30:56-55:28 for DIVIDE/CALCULATE/IF patterns

<https://www.youtube.com/watch?v=ID7TvkoQ6rY>

<https://learn.microsoft.com/en-us/credentials/certifications/resources/study-guides/pl-300>

After watching/opening: return to this lesson and reproduce the worked example from your own file/query/report. Save course-task evidence, not a screenshot of the source.

7. Follow with me once

Teacher demonstration - do this once with me, then close the example before your fresh task.

- 1 Base [Sales] stays trusted.
- 2 Define [Sales All Categories] by removing only the intended Category filter while retaining other relevant filters such as Region/Date.
- 3 Define [Category Contribution %] = $\text{DIVIDE}([\text{Sales}], [\text{Sales All Categories}])$. State whether Product-level filters should also be removed.
- 4 Define [Target] from the higher-grain target fact through valid dimensions, then $[\text{Variance}] = [\text{Sales}] - [\text{Target}]$ and $[\text{Variance \%}] = \text{DIVIDE}([\text{Variance}], [\text{Target}])$.
- 5 For a closing inventory balance, do not SUM daily closing balances across dates; define the last relevant date and return the balance for that date.

8. What you should see / be able to say

Expected result

A small family of business measures with contracts that define total/reference/time behavior.

9. Why it worked

Reasoning

Each complex result branches from validated base measures and modifies only the context needed by the business definition. Professional upgrade: Advanced measures are governed business logic. Keep definitions near the measure catalog and validate behavior at totals/subtotals, not only detail rows.

10. Common beginner mistakes

1. Using ALL on the entire model/table without understanding which filters disappear. | 2. Summing snapshots across time. | 3. Dividing by a target at incompatible grain. | 4. Creating one giant measure that hides every intermediate result.

11. If you get stuck - recovery ladder

Recovery

Break the final measure into base, denominator/reference, delta and final ratio. Put them side by side in a test matrix. Save the genuine attempt before review.

12. Now you do a small version alone

Fresh independent task

Fresh case: team contribution to resolved tickets, variance to staffing target, and end-of-week backlog balance.

13. Validate your result

Validation

Reference total, target grain and end-of-period date are independently visible and reconciled.

14. Teach it back in your own words

Explain without reading

Explain why the word 'total' must be defined in contribution %.

15. Protected review + changed fresh retry

Attempt protection

Attempt first. Changed retry uses margin share + monthly target + closing customer balance.

16. Evidence / mastery + exact next route

Mastery evidence

Measure family + contracts + subtotal/total tests + explain-back. VIDEO SUPPORT: Chandoo DAX masterclass - 30:56-55:28 for DIVIDE/CALCULATE/IF patterns <https://learn.microsoft.com/en-us/credentials/certifications/resources/study-guides/pl-300> After saving evidence, continue to DAX10.

DAX10 - Debugging incorrect measures

1. Why it matters

Changing random DAX until a visual 'looks right' can create a new hidden error. Senior debugging identifies the first wrong intermediate result or context assumption.

2. What you are learning

Reduce a problem visual; test base measures; expose variables/intermediates; inspect context in a tiny matrix; use DAX Query View/Performance Analyzer when appropriate; rerun total/subtotal controls after repair.

3. Prerequisite / what you already know

DAX1-DAX9.

4. Picture it in your head

Mental model

Measure autopsy: freeze one wrong cell, list its active filters, test the base measure, expose variables/intermediates, then compare detail/subtotal/total. Repair the first point where expected and observed behavior diverge.

5. New words before we use them

DAX Query View: Power BI Desktop capability for writing/evaluating DAX queries against the semantic model, useful for exploration/debugging. Subtotal behavior: How a measure evaluates at broader context rather than summing visible child rows. Regression check: Re-running previously passing controls after a change to ensure nothing else broke.

6. Watch / visual source

Before/after measure + debug log + regression cases + explain-back. VIDEO SUPPORT: Chandoo DAX masterclass - 58:20-end optional ACMBU practice mindset

<https://www.youtube.com/watch?v=ID7TvkoQ6rY>

<https://learn.microsoft.com/en-us/credentials/certifications/resources/study-guides/pl-300>

After watching/opening: return to this lesson and reproduce the worked example from your own file/query/report. Save course-task evidence, not a screenshot of the source.

7. Follow with me once

Teacher demonstration - do this once with me, then close the example before your fresh task.

- 1 Record one exact wrong cell and expected business meaning.
- 2 Put [Sales], denominator/reference measure and final % into a tiny matrix.
- 3 Inspect which filters reach the denominator at detail and total levels.
- 4 If the measure uses VAR, temporarily return each intermediate. If useful, use DAX Query View to inspect model results without the full dashboard.
- 5 Repair only the faulty context expression and rerun detail, subtotal, grand-total and slicer checks.

8. What you should see / be able to say

Expected result

A debug log with one reproducible failure, first wrong intermediate/context, root cause, repair and regression evidence.

9. Why it worked

Reasoning

The error is isolated before the expression is changed. Professional upgrade: Keep a small DAX QA page/query set with base/intermediate/control measures. Debugging evidence is part of maintainable BI work.

10. Common beginner mistakes

1. Editing five functions at once. | 2. Assuming grand total should equal sum of visible percentages. | 3. Testing only one slicer state. | 4. Using Performance Analyzer to debug semantic correctness.

11. If you get stuck - recovery ladder

Recovery

Start from trusted base measure. Add one derived layer at a time in a tiny table. Stop at the first wrong value. Save the genuine attempt before review.

12. Now you do a small version alone

Fresh independent task

Fresh case: rolling 4-week measure is correct by week but wrong under Team slicer. Find the first incorrect context assumption.

13. Validate your result

Validation

Original failing case plus at least three regression cases pass after repair.

14. Teach it back in your own words

Explain without reading

Explain why a total can correctly recalculate a measure instead of summing child-row results.

15. Protected review + changed fresh retry

Attempt protection

Attempt first. Changed retry uses a prior-period bug instead of contribution %.

16. Evidence / mastery + exact next route

Mastery evidence

Before/after measure + debug log + regression cases + explain-back. VIDEO SUPPORT: Chandoo DAX masterclass - 58:20-end optional ACMBU practice mindset <https://learn.microsoft.com/en-us/credentials/certifications/resources/study-guides/pl-300> After saving evidence, continue to DAX11.

DAX11 - Report storytelling and executive design

1. Why it matters

2. What you are learning

Design a decision-first report page: headline, context/comparison, drivers/exceptions, recommendation, definitions/limitations; choose visuals that answer a question rather than decorate.

3. Prerequisite / what you already know

Validated model/measures. Basic Power BI visuals from Course 1.

4. Picture it in your head

Mental model

Decision funnel: headline KPI says what happened, trend says when, driver view shows where, exception view shows risk, and recommendation says what to do. Every visual earns its place by answering a decision question.

5. New words before we use them

Visual hierarchy: The order in which layout/size/position guides attention. Narrative: A sequence connecting evidence to interpretation and action. Exception visual: A visual emphasizing unusual/decision-relevant cases rather than showing every category equally.

6. Watch / visual source

Wireframe + report screenshot + decision narrative + metric references + explain-back. VIDEO: intentionally book-led for this lesson

<https://learn.microsoft.com/en-us/credentials/certifications/resources/study-guides/pl-300>

After watching/opening: return to this lesson and reproduce the worked example from your own file/query/report. Save course-task evidence, not a screenshot of the source.

7. Follow with me once

Teacher demonstration - do this once with me, then close the example before your fresh task.

- 1 Start with the decision: where should leadership intervene this month?
- 2 Top row: Revenue, Margin, Return Rate versus approved comparison/target. Do not overload with 12 cards.
- 3 Second layer: one trend plus the most important driver split; show context, not every dimension.
- 4 Exception layer: regions/products that materially explain the gap.
- 5 Close with a short recommendation and a visible definition/limitation note when it changes interpretation.

8. What you should see / be able to say

Expected result

One page where a new reader can state the main change and likely action within about a minute.

9. Why it worked

Reasoning

The page follows the decision path from status to explanation to action. Professional upgrade: Senior report design optimizes decision clarity, consistency and trust. Extra drill paths can exist without competing with the executive surface.

10. Common beginner mistakes

1. Using every visual type learned. | 2. Decorating before defining the decision. | 3. Showing percentages without denominator/context. | 4. Hiding caveats in tiny tooltip-only text.

11. If you get stuck - recovery ladder

Recovery

Delete everything except the decision question. Add only one visual needed to answer each of: What happened? Why? Where? What next? Save the genuine attempt before review.

12. Now you do a small version alone

Fresh independent task

Fresh case: executive support page for backlog and SLA. Build a five-block wireframe before Desktop formatting.

13. Validate your result

Validation

Ask a reviewer to state the decision/action after 60 seconds; verify every visual has a question and metric definition.

14. Teach it back in your own words

Explain without reading

Explain why a beautiful dashboard can still be a poor executive report.

15. Protected review + changed fresh retry

Attempt protection

Attempt first. Changed retry uses customer retention instead of revenue.

16. Evidence / mastery + exact next route

Mastery evidence

Wireframe + report screenshot + decision narrative + metric references + explain-back. VIDEO: intentionally book-led for this lesson
<https://learn.microsoft.com/en-us/credentials/certifications/resources/study-guides/pl-300> After saving evidence, continue to DAX12.

DAX12 - Drillthrough, tooltips, bookmarks and interaction

1. Why it matters

Interactivity should help a user answer a follow-up question, not create a maze. Bookmarks, tooltips and drillthrough each solve different interaction needs.

2. What you are learning

Choose between tooltip, drillthrough, bookmark/navigation and visual interactions; preserve context deliberately; test back/reset paths; avoid hiding essential information in hover-only content.

3. Prerequisite / what you already know

DAX11 report hierarchy and basic Power BI report editing.

4. Picture it in your head

Mental model

Round-trip navigation: the user leaves an overview with known context, reaches detail, understands what state changed, and can return/reset without losing orientation. Tooltip, drillthrough and bookmark state are UX contracts, not decoration.

5. New words before we use them

Drillthrough: Navigation to a target page filtered by a selected data point/context. Bookmark: Saved report state that can capture page, filters, slicers, selection/visibility and other state. Tooltip: Contextual information shown around a visual/data point, depending on tooltip type/settings.

6. Watch / visual source

Interaction map + screenshots + three navigation tests + explain-back. VIDEO: intentionally book-led for this lesson

<https://learn.microsoft.com/en-us/power-bi/create-reports/desktop-bookmarks>

After watching/opening: return to this lesson and reproduce the worked example from your own file/query/report. Save course-task evidence, not a screenshot of the source.

7. Follow with me once

Teacher demonstration - do this once with me, then close the example before your fresh task.

- 1 Use overview page to show customer segment exceptions.
- 2 Tooltip: add secondary context such as order count or margin; do not place the only critical warning there.
- 3 Drillthrough: create Customer Detail page with CustomerID/Customer field in drillthrough filters and retain the back path.
- 4 Bookmark: use a clearly named state for an explanatory panel or navigation pattern; update/test it after slicer/layout changes.
- 5 Test interactions from multiple source visuals and reset/back behavior before publishing.

8. What you should see / be able to say

Expected result

A report interaction map where each feature has one clear purpose and users can always understand how to return/reset.

9. Why it worked

Reasoning

Interaction choice follows the user's next question rather than feature novelty. Professional upgrade: Treat interaction state as report logic. Changes to slicers/visuals can invalidate saved bookmark expectations, so interaction QA belongs in release testing.

10. Common beginner mistakes

1. Putting essential information only in a tooltip. | 2. Creating hidden drillthrough pages with no discovery cue. | 3. Bookmarks capturing unintended slicer/filter state. | 4. Cross-highlighting every visual even when it confuses the story.

11. If you get stuck - recovery ladder

Recovery

Write the user's follow-up question. Pick only one interaction needed to answer it. Test from a clean report state. Save the genuine attempt before review.

12. Now you do a small version alone

Fresh independent task

Fresh case: support overview -> Team detail drillthrough; one diagnostic tooltip; one bookmark-controlled definition panel.

13. Validate your result

Validation

Source context arrives correctly, back/reset works, bookmark state is intentional, and key info remains accessible without hover.

14. Teach it back in your own words

Explain without reading

Explain the difference between tooltip, drillthrough and bookmark in one sentence each.

15. Protected review + changed fresh retry

Attempt protection

Attempt first. Changed retry uses Product overview -> Product detail.

16. Evidence / mastery + exact next route

Mastery evidence

Interaction map + screenshots + three navigation tests + explain-back. VIDEO: intentionally book-led for this lesson
<https://learn.microsoft.com/en-us/power-bi/create-reports/desktop-bookmarks> After saving evidence, continue to DAX13.

DAX13 - Accessibility and usability

1. Why it matters

A report is not usable if important conclusions depend on color alone, tiny text, mouse-only hover, or confusing navigation. Accessibility improves clarity for everyone.

2. What you are learning

Use meaningful titles/labels, contrast, non-color cues, alt text, logical tab/focus order, accessible tables/sort order, keyboard-friendly navigation and visible critical information.

3. Prerequisite / what you already know

DAX11/DAX12 report design and interaction.

4. Picture it in your head

Mental model

Alternate channel: every critical meaning must survive without color, hover, mouse precision or perfect vision. Contrast, labels, alt text, keyboard order and visible instructions must carry the same business message through different channels.

5. New words before we use them

Alt text: Text description that helps assistive technology communicate the purpose/content of an object. Focus order: The sequence in which interactive elements receive keyboard focus. Non-color cue: Text, icon, shape, pattern or position that communicates meaning in addition to color.

6. Watch / visual source

Accessibility audit + before/after screenshots + keyboard/focus notes + explain-back. VIDEO: intentionally book-led for this lesson

<https://learn.microsoft.com/en-us/power-bi/create-reports/desktop-accessibility-creating-reports>

After watching/opening: return to this lesson and reproduce the worked example from your own file/query/report. Save course-task evidence, not a screenshot of the source.

7. Follow with me once

Teacher demonstration - do this once with me, then close the example before your fresh task.

- 1 Replace red/green-only status with label/icon plus color.
- 2 Write meaningful visual titles and alt text based on the decision/question, not 'Chart 1'.
- 3 Check logical tab/focus order and keyboard navigation, especially bookmark/page navigator controls.
- 4 Do not make a critical limitation available only in a report tooltip; keep essential content visible or otherwise accessible.
- 5 Set intentional visual sort order because accessible data views follow the report's sort.

8. What you should see / be able to say

Expected result

An accessibility checklist with corrected page evidence and no critical insight dependent solely on color/hover.

9. Why it worked

Reasoning

The same business meaning is available through multiple perceivable/navigation routes. Professional upgrade: Accessible design belongs in templates and release checklists so it is not postponed to the end.

10. Common beginner mistakes

1. Using alt text that repeats a generic visual type. | 2. Low contrast decorative text. | 3. Tooltip-only definitions. | 4. Keyboard focus jumping in a visually illogical order.

11. If you get stuck - recovery ladder

Recovery

Temporarily ignore color and mouse hover. Can you still identify status, navigate and read the key conclusion? Repair the first failure. Save the genuine attempt before review.

12. Now you do a small version alone

Fresh independent task

Fresh case: audit your DAX11 page for color-only meaning, titles, alt text, focus order, sort and tooltip dependency.

13. Validate your result

Validation

Complete keyboard/navigation check and record at least one before/after usability improvement.

14. Teach it back in your own words

Explain without reading

Explain why accessibility is a report-quality requirement, not an optional polish step.

15. Protected review + changed fresh retry

Attempt protection

Attempt first. Changed retry audits a drillthrough page instead of the executive overview.

16. Evidence / mastery + exact next route

Mastery evidence

Accessibility audit + before/after screenshots + keyboard/focus notes + explain-back. VIDEO: intentionally book-led for this lesson
<https://learn.microsoft.com/en-us/power-bi/create-reports/desktop-accessibility-creating-reports> After saving evidence, continue to DAX14.

DAX14 - Row-level security

1. Why it matters

RLS is a data-access control, not a visual filter. A report can look correct for the author while exposing too much or too little data to a consumer if role/model behavior is not tested.

2. What you are learning

Understand static versus dynamic RLS; prefer filtering appropriate dimension/mapping tables where the model supports it; use USERPRINCIPALNAME for dynamic patterns; validate roles/users before release.

3. Prerequisite / what you already know

C2-03 relationship/filter propagation, DAX context basics.

4. Picture it in your head

Mental model

Airport gate: identity is the credential, the mapping/role rule selects allowed dimension rows, relationships propagate the restriction to facts, and View as / real-user tests prove what each identity can and cannot see. Test deny/edge cases too.

5. New words before we use them

Row-level security: Model rules restricting which data rows a user can access. Static RLS: Role rule fixed to a defined value/set such as Region = West.

Dynamic RLS: Role logic using user identity/mapping so one role filters differently by user. USERPRINCIPALNAME: DAX function commonly used to obtain the signed-in user's UPN for dynamic RLS patterns.

6. Watch / visual source

RLS mapping + role rule + test matrix + visible-total controls + explain-back. VIDEO: intentionally book-led for this lesson

<https://learn.microsoft.com/en-us/fabric/security/service-admin-row-level-security>

After watching/opening: return to this lesson and reproduce the worked example from your own file/query/report. Save course-task evidence, not a screenshot of the source.

7. Follow with me once

Teacher demonstration - do this once with me, then close the example before your fresh task.

- 1 Create UserRegion mapping with UserUPN and Region; ensure mapping grain/duplicates are intentional.
- 2 Define dynamic role filter on the mapping/user dimension using USERPRINCIPALNAME() under a model that propagates filters safely to Region/facts.
- 3 Do not put a security rule on a random fact table just because it returns the desired screenshot; design for clear filter propagation.
- 4 Use View as role / test user behavior before publishing, then validate again in the service when available.
- 5 Test no-access, single-region and multi-region users plus an unknown user case according to the approved policy.

8. What you should see / be able to say

Expected result

An RLS design with role/mapping contract and evidence that different test identities see exactly the allowed data.

9. Why it worked

Reasoning

User identity restricts a governed mapping/dimension path which then propagates to facts. Professional upgrade: Current Power BI guidance requires role validation and careful service/workspace behavior. Security changes need explicit regression tests before release.

10. Common beginner mistakes

1. Treating RLS as report-page filtering. | 2. Forgetting duplicate/multi-region user mappings. | 3. Assuming Desktop author view proves consumer security. | 4. Ignoring workspace/service role implications.

11. If you get stuck - recovery ladder

Recovery

12. Now you do a small version alone

Fresh independent task

Fresh case: regional managers can see one/multiple assigned regions; executives see all via a separately governed role/policy.

13. Validate your result

Validation

Test at least four identities/role cases and reconcile visible totals to expected allowed rows.

14. Teach it back in your own words

Explain without reading

Explain why hiding a visual is not row-level security.

15. Protected review + changed fresh retry

Attempt protection

Attempt first. Changed retry uses Team access rather than Region.

16. Evidence / mastery + exact next route

Mastery evidence

RLS mapping + role rule + test matrix + visible-total controls + explain-back. VIDEO: intentionally book-led for this lesson
<https://learn.microsoft.com/en-us/fabric/security/service-admin-row-level-security> After saving evidence, continue to DAX15.

DAX15 - Workspaces, sharing and lifecycle basics

1. Why it matters

A PBIX is not the production system. Once content is shared, workspace roles, semantic-model permissions, apps, refresh and republishing can affect other reports/users.

2. What you are learning

Understand workspace roles at a basic level, publish semantic model/report, distinguish builder workspace access from app consumption, plan refresh/gateway needs, endorse content when appropriate, and assess impact before replacing shared models.

3. Prerequisite / what you already know

DAX14 security and C2-03 model/refresh concepts. Power BI Service access is helpful but not required for conceptual completion.

4. Picture it in your head

Mental model

Release pipeline: Desktop authoring -> workspace ownership -> semantic model/report/app -> access/security -> refresh/gateway -> monitoring/change rollback. Publish is the start of operating the BI product, not the end of development.

5. New words before we use them

Workspace: Collaborative Power BI/Fabric area where teams create/manage content. Power BI app: Packaged/distributed content experience for business consumers. Semantic model permission: Permission such as Read/Build/Reshare/Write affecting what users can do with a semantic model. Endorsement: Promotion/certification signals that content is recommended/trusted under governance.

6. Watch / visual source

Release/lifecycle checklist + role/access matrix + refresh owner + change-impact note + explain-back. VIDEO: intentionally book-led for this lesson

<https://learn.microsoft.com/en-us/power-bi/collaborate-share/collaborate-share-overview>

After watching/opening: return to this lesson and reproduce the worked example from your own file/query/report. Save course-task evidence, not a screenshot of the source.

7. Follow with me once

Teacher demonstration - do this once with me, then close the example before your fresh task.

- 1 Identify builder roles versus consumers. Workspace access is for collaboration; app/share pathways are often better for broad consumption.
- 2 Publish the report/semantic model to the approved workspace and confirm expected items exist.
- 3 Configure/verify data-source credentials and scheduled refresh requirements where the model imports data; determine whether a gateway is required for the source.
- 4 Choose distribution: app for polished audience distribution, direct share only when justified. Record semantic-model Build/other permissions deliberately.
- 5 Before republishing a changed semantic model, inspect downstream impact and rerun security/refresh/report QA. Record version/change notes.

8. What you should see / be able to say

Expected result

A simple release checklist covering workspace, permissions, publish/distribution, refresh, security and change impact.

9. Why it worked

Reasoning

The lifecycle separates development, collaboration, consumption and operation instead of treating Publish as the final step. Professional upgrade: Licensing/capacity/tenant policies change what options are available. The analyst should document the chosen distribution and ownership model rather than hard-code assumptions from another organization.

10. Common beginner mistakes

1. Giving everyone high workspace roles because sharing is easier. | 2. Assuming app access equals workspace edit access. | 3. Forgetting credentials/gateway/scheduled refresh after publish. | 4. Replacing a shared semantic model without impact analysis.

11. If you get stuck - recovery ladder

Recovery

List who builds, who consumes, what refreshes, who owns credentials/model, and what downstream items depend on it. Choose the minimum appropriate access route. Save the genuine attempt before review.

12. Now you do a small version alone

Fresh independent task

Fresh case: publish a monthly operations model/report for 3 builders and 50 consumers; design workspace/app/refresh/permission handoff.

13. Validate your result

Validation

Every actor has an intentional role/access path; refresh owner and failure response are named; model replacement has an impact/checklist step.

14. Teach it back in your own words

Explain without reading

Explain workspace versus app using 'build room' versus 'consumer package' without oversimplifying permissions.

15. Protected review + changed fresh retry

Attempt protection

Attempt first. Changed retry uses a shared semantic model consumed by reports in another workspace.

16. Evidence / mastery + exact next route

Mastery evidence

Release/lifecycle checklist + role/access matrix + refresh owner + change-impact note + explain-back. VIDEO: intentionally book-led for this lesson <https://learn.microsoft.com/en-us/power-bi/collaborate-share/collaborate-share-overview> After saving evidence, complete the module mastery checklist and follow the Gate A / completion route.

C2-04 mastery checklist + quick reference

Use after the first full pass; do not replace Power BI practice with reading

FIFTEEN QUESTIONS

DAX1 What filters reach this cell? | DAX2 Is there row context, filter context, or both? | DAX3 Which filter does CALCULATE add/replace? | DAX4 What table/grain does the iterator visit? | DAX5 Where does row context become filter context? | DAX6 Is the Date table/role valid? | DAX7 Are comparison/window periods comparable? | DAX8 Which variable/intermediate is wrong? | DAX9 Is total/target/snapshot context defined? | DAX10 What is the first wrong intermediate/context? | DAX11 What decision does the page support? | DAX12 What question does each interaction answer? | DAX13 Can users access meaning without color/hover/mouse? | DAX14 Which rows should each identity see? | DAX15 Who builds, consumes, refreshes and owns the shared model?

BLOCK MASTERY IF

Measure works only in one visual state; totals/subtotals untested; wrong Date role; hidden denominator/ALL behavior; accessibility-critical content only in tooltip/color; RLS untested; workspace/access route over-permissioned without justification; publish/refresh ownership unknown.

FAST DEBUG

Simplify visual -> show base/intermediate measures -> trace filters -> test one dimension -> detail/subtotal/grand total -> changed slicer -> repair one cause -> regression tests. For UX/security, test navigation/focus/identities from clean state.

COURSE POSITION

Optional Senior/BI depth. Direct DA-to-DE learners can use C2B/C3 without C2-04. PL-300 alignment is documented later but does not make certification a prerequisite.

C2-04 source / video registry

Current DAX/report/security/lifecycle support; internal teaching remains sufficient

DAX1-3 - Chandoo - Learn 80% of DAX in an Hour

<https://www.youtube.com/watch?v=ID7TVkoQ6rY>

19:24-46:34 context/CALCULATE

DAX1-2 - Microsoft Learn - DAX overview

<https://learn.microsoft.com/en-us/dax/dax-overview>

Row/filter/query context

DAX3/5 - Microsoft Learn - CALCULATE / modify filter context

<https://learn.microsoft.com/en-us/dax/calculate-function-dax>

Filter modification + context transition

DAX4 - Microsoft Learn - SUMX

<https://learn.microsoft.com/en-us/dax/sumx-function-dax>

Iterator reference

DAX6 - Microsoft Learn - Time intelligence in Power BI

<https://learn.microsoft.com/en-us/power-bi/transform-model/desktop-time-intelligence>

Current time-intelligence guidance

DAX7 - Microsoft Learn - DATEADD

<https://learn.microsoft.com/en-us/dax/dateadd-function-dax>

Current DATEADD behavior

DAX8 - Microsoft Learn - DAX variables best practices

<https://learn.microsoft.com/en-us/dax/best-practices/dax-variables>

VAR readability/performance

DAX9-10 - PL-300 2026 study guide

<https://learn.microsoft.com/en-us/credentials/certifications/resources/study-guides/pl-300>

DAX calculations + Performance Analyzer/DAX Query View

DAX12 - Microsoft Learn - report bookmarks

<https://learn.microsoft.com/en-us/power-bi/create-reports/desktop-bookmarks>

Bookmark state

DAX12 - Microsoft Learn - visual tooltips / drillthrough

<https://learn.microsoft.com/en-us/power-bi/create-reports/desktop-custom-tooltips>

Tooltips + drill actions

DAX13 - Microsoft Learn - Power BI accessibility

<https://learn.microsoft.com/en-us/power-bi/create-reports/desktop-accessibility-creating-reports>

Alt text, keyboard, tooltip caution

DAX14 - Microsoft Learn - Row-level security

<https://learn.microsoft.com/en-us/fabric/security/service-admin-row-level-security>

Static/dynamic RLS + validation

DAX15 - Microsoft Learn - Collaborate/share overview

<https://learn.microsoft.com/en-us/power-bi/collaborate-share/collaborate-share-overview>

Workspace/share/app overview

DAX15 - Microsoft Learn - Workspace roles

<https://learn.microsoft.com/en-us/power-bi/collaborate-share/service-roles-new-workspaces>

Admin/Member/Contributor/Viewer roles

DAX15 - Microsoft Learn - Scheduled refresh

<https://learn.microsoft.com/en-us/power-bi/connect-data/refresh-scheduled-refresh>

Refresh/gateway operational basics

VERIFICATION RULE

Sources above were rechecked on the web 2026-09-10. External media supports visualization/current UI; the book and supplied practice control assessment logic.